Client Interview Questions(karat)

18 March 2026 03:03 PM

//Coding question// remove duplicates from a list

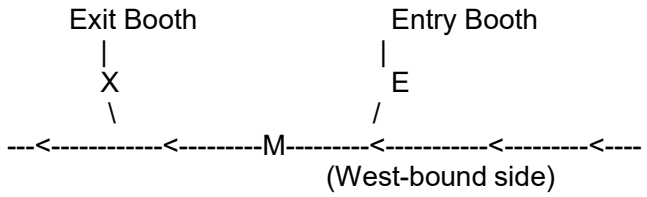
```
List<Integer > myList = Arrays.asList(10,1,3,3,2,5,6,6);
List<Integer> uniqueList =myList.stream().distint().collect(Collectors.toList);
```

1. serialization
2. in multithreading what is deadlock
3. what happens if we don't override equals and hashcode both
4. how will you create the custom exception in springboot
5. what is parent class in Java
6. how to create a procedure in database
7. What will be output of below program.
8. Which memory is garbage collected

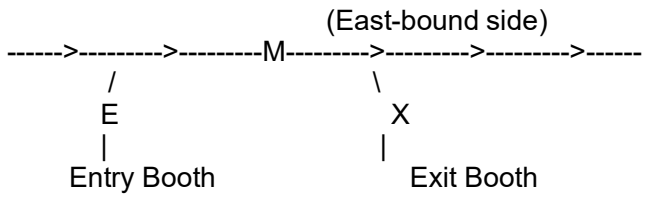
```
package com.org.arpit.java2blog;
import java.util.HashSet;
public class Employee {
public String name;
@Override
public int hashCode() {
return 31;
}
public static void main(String args[]) {
Employee employeeOne = new Employee();
Employee employeeTwo = new Employee();
employeeOne.name = "John";
employeeTwo.name = "John";
HashSet employeeSet = new HashSet();
employeeSet.add(employeeOne);
employeeSet.add(employeeTwo);
System.out.println(employeeSet.size());
}
}
```

We are writing software to analyze logs for toll booths on a highway.
This highway is a divided highway with limited access;
the only way on to or off of the highway is through a toll booth.

There are three types of toll booths:
* ENTRY (E in the diagram) toll booths, where a car goes through a booth as it enters the highway.
* EXIT (X in the diagram) toll booths, where a car goes through a booth as it exits the highway.
* MAINROAD (M in the diagram), which have sensors that record a license plate as a car drives through at full speed.



==========



For our first task:
1-1) Read through and understand the code and comments below. Feel free to run the code and tests.
1-2) The tests are not passing due to a bug in the code. Make the necessary changes to LogEntry to fix the bug.
*/

/*
We are interested in how many people are using the highway, and so we would like to count how many complete journeys are taken in the log file.

A complete journey consists of:
* A driver entering the highway through an ENTRY toll booth.
* The driver passing through some number of MAINROAD toll booths (possibly 0).
* The driver exiting the highway through an EXIT toll booth.

For example, the following excerpt of log lines contains complete journeys for the cars with JOX304 and THX138:

.
.
.
90750.191 JOX304 250E ENTRY
91081.684 JOX304 260E MAINROAD
91082.101 THX138 110E ENTRY
91483.251 JOX304 270E MAINROAD
91873.920 THX138 120E MAINROAD
91874.493 JOX304 280E EXIT
.
.
91982.102 THX138 290E EXIT
92301.302 THX138 300E ENTRY
92371.302 THX138 310E EXIT
.

→ This log contains 3 complete journeys:
- JOX304: 1 journey
- THX138: 2 journeys

You may assume that the log only contains complete journeys, and there are no missing entries.

2-1) Write a function in LogFile named countJourneys() that returns how many complete journeys there are in the given LogFile.
*/

/*

We would like to catch people who are driving at unsafe speeds on the highway. To help us do that, we would like to identify journeys where a driver does either of the following:
* Drive 130 km/h or greater in any individual 10km segment of tollway.
* Drive 120 km/h or greater in any two 10km segments of tollway.

For example, consider the following journey:
1000.000 TST002 270W ENTRY
1275.000 TST002 260W EXIT

In this case, the driver of TST002 drove 10 km in 275 seconds. We can calculate that this driver drove an average speed of ~130.91km/hr over this segment:

10 km * 3600 sec/hr
---------- = 130.91 km/hr
275 sec

Note that:
* A license plate may have multiple journeys in one file, and if they drive at unsafe speeds in both journeys, both should be counted.
* We do not mark speeding if they are not on the highway (i.e. for any driving between an EXIT and ENTRY event).
* Speeding is only marked once per journey. For example, if there are 4 segments 120km/h or greater, or multiple segments 130km/h or greater, the journey is only counted once.

3-1) Write a function catchSpeeders in LogFile that returns a collection of license plates that drove at unsafe speeds during a journey in the LogFile.
If the same license plate drives at unsafe speeds during two different journeys, the license plate should appear twice (once for each journey they drove at unsafe speeds).
*/

/*
Also we would like to catch people who are driviing at unsafe speed on the highway
To help us do tha twe would like to identify journeys where driver does either of the following
*Driver 130km/h or greater in any individual 10 km segment of tollway
*Driver 120km/h or greater in any two 10km segment of tollway

for xample
1000.000 TST002 270W ENTRY
1275.000 TST002 260W EXIT
in this case the driver drove 10km in 275 seconds. we can calculate that this drivers drove an average speed of 130.9/km/h over this segment

10km * 3600sec/hr
----------= 130.91km/hr
275 sec

Note that
A license plat may have multiple journeys in one file, and if they drive at unsafe speed in both journeys, both should be counte
We dn't mark speeding if they are not on the highway(i.e for any driving between an EXIT and ENTRY event)
Speeding is only marked once per journey. For example , if there are 4 segment 120km/h or greater, or multiple segments 130km/h or greater, they jouney is only counted once
*/

@SBA

@RequestMapping(/Employee)
@RestController
public class EmployeeApplication{

@Autowired

@GetMapping("/")
Public String getEmployeeDetails(){

}

}

-->
Service

@Service

--->

@Re

@SBTest
@

@Test
public void saveData(){

// creating a object to save
//saving into a db (inserting)
//Employee service save

}

List<Integer> myList = 6,8,10,4;

//2nd largest

myList.stream().sorted(Comparator.reverseOrder()).skip(1).findFirst();;

String str = "abc";
WAP to illustrate maximum possibility of permutation and combination of the string

a,b,c
a.b,c

List<Integer> listInt = 6,8,10,4,1,3,2;

Find the sum of two number whose sum is 5

```
for(listInt ){

      for(){

}


}
```

Disadvantages of mS
When you will go with monolithic and mS
equals and hashcode

out of memory error

Exception handling

how to do chunk processing from db to file
caching

Different design pattern

SOLID principle

Multithreading:
Scenario where you used multithreading.
executer framework --> different parameters

Different ways to synchronized
volatile

If client hit a request and dont want to wait for the response.
Asyn call.
Then how to notify the client on successful/ failure?
Suppose we are sending 1000 message over kafka topic, there are failure with 100 messages, then how will you approach?
(And we dont want to process one by one for failure messages)

Example for the failure of Liskov principles

ArrayList vs LinkedList (When to go with ArrayList and LinkedList)
HashMap internal working.

Which you will prefer, code using prepared statement or code using JPA repository ?
Performance wise which is better?